DSA Streak: Daily LeetCode in Java

2026-11-14 | 5 problems

#	Problem	Difficulty	Topic
1	Minimum Positive Sum Subarray	Easy	Array, Sliding Window
2	Invert Binary Tree	Easy	Tree, Depth-First Search
3	Top K Frequent Elements	Medium	Array, Hash Table
4	Number of Islands	Medium	Array, Depth-First Search
5	Minimum Window Substring	Hard	Hash Table, String

1. Minimum Positive Sum Subarray

Easy | #3364 | Array, Sliding Window, Prefix Sum | Problem of the Day
https://leetcode.com/problems/minimum-positive-sum-subarray/

Problem

Given an integer array nums and two integers l and r, look at every subarray whose length is between l and r (inclusive). Among those whose sum is strictly greater than 0, return the smallest sum. If none exist, return -1.

Examples

Example 1	
Input	`nums = [3, -2, 1, 4], l = 2, r = 3`
Output	`1`
Why	Valid positive subarrays are [3,-2]=1, [1,4]=5, [3,-2,1]=2, [-2,1,4]=3. The smallest is 1.

Example 2	
Input	`nums = [-2, 2, -3, 1], l = 2, r = 3`
Output	`-1`
Why	No subarray of length 2 or 3 has a sum greater than 0, so the answer is -1.

Example 3	
Input	`nums = [1, 2, 3, 4], l = 2, r = 4`
Output	`3`
Why	[1,2] has length 2 and sum 3, which is the smallest positive sum possible.

Intuition

n is at most 100, so we can afford to look at every subarray. Fix a start index i and extend the end j one step at a time while keeping a running sum, so each new subarray costs O(1) instead of re-adding all its elements. Only lengths from l to r count, and only positive sums are candidates.

Approach

1. Set ans = Integer.MAX_VALUE.
2. For each start i, reset sum = 0.
3. Move j from i forward while the length j - i + 1 is at most r, adding nums.get(j) to sum.
4. If the length is at least l and sum > 0, set ans = min(ans, sum).
5. After all starts, return -1 if ans was never updated, else ans.

Dry run on Example 1

- nums = [3,-2,1,4], l = 2, r = 3, ans = MAX
- i=0: j=0 sum=3 (len 1, too short); j=1 sum=1 (len 2, positive) ans=1; j=2 sum=2 (len 3) ans stays 1; j=3 would be len 4 > r, stop
- i=1: j=1 sum=-2; j=2 sum=-1 (not positive); j=3 sum=3 (len 3) ans=min(1,3)=1
- i=2: j=2 sum=1 (len 1, too short); j=3 sum=5 (len 2) ans stays 1
- i=3: j=3 sum=4 (len 1, too short)
- ans = 1 is not MAX, return 1

Java solution

```
import java.util.*;

class Solution {
    public int minimumSumSubarray(List<Integer> nums, int l, int r) {
```

```
        int n = nums.size();
        int ans = Integer.MAX_VALUE;
        for (int i = 0; i < n; i++) {
            int sum = 0;
            for (int j = i; j < n && j - i + 1 <= r; j++) {
                sum += nums.get(j);
                if (j - i + 1 >= l && sum > 0) ans = Math.min(ans, sum);
            }
        }
        return ans == Integer.MAX_VALUE ? -1 : ans;
    }
}
```

Complexity

Time: O(n * r), at most O(n^2) = 10^4 steps for n = 100 Space: O(1)

Edge cases handled

- All sums zero or negative: ans stays MAX, return -1
- l = r: only one window size is checked
- Single element with l = r = 1: return it if positive, else -1
- Zero sums do not count because the sum must be strictly greater than 0

Pattern to remember

When n is small, fix the start and grow the end with a running sum to check every subarray in O(n^2) without an extra loop for the sum.

2. Invert Binary Tree

Easy | #226 | Tree, Depth-First Search, Breadth-First Search, Binary Tree
https://leetcode.com/problems/invert-binary-tree/

Problem

Given the root of a binary tree, mirror it: swap the left and right child of every node. Return the root.

Examples

Example 1	
Input	`root = [4,2,7,1,3,6,9]`
Output	`[4,7,2,9,6,3,1]`
Why	Every node's children are swapped, so the tree becomes its mirror image.

Example 2	
Input	`root = [2,1,3]`
Output	`[2,3,1]`
Why	The root's two children 1 and 3 swap places.

Example 3	
Input	`root = []`
Output	`[]`
Why	An empty tree stays empty.

Intuition

A tree is mirrored if both of its subtrees are mirrored and then placed on the opposite sides. That is a natural recursion: solve the left and right subtrees, then swap them at the current node.

Approach

1. If root is null, return null.
2. Recursively invert the left subtree and save it in l.
3. Set root.left to the inverted right subtree.
4. Set root.right to l.
5. Return root.

Dry run on Example 1

- invertTree(4): l = invertTree(2)
- invertTree(2): l = invertTree(1) = 1 (leaf, children null), root.left = invertTree(3) = 3, root.right = 1, so 2 now has children 3, 1
- back at 4: l = node 2 (already inverted)
- root.left = invertTree(7): inside, l = 6, root.left = 9, root.right = 6, so 7 now has children 9, 6
- root.right = l = node 2
- Tree is now 4 -> (7 -> 9, 6), (2 -> 3, 1), level order [4,7,2,9,6,3,1]

Java solution

```
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) return null;
        TreeNode l = invertTree(root.left);
        root.left = invertTree(root.right);
        root.right = l;
```

```
        return root;
    }
}
```

Complexity

Time: O(n), each node is visited once Space: O(h) recursion stack, h = height (O(n) worst case for a skewed tree)

Edge cases handled

- Empty tree: return null
- Single node: nothing to swap
- Skewed tree (only left children) becomes only right children
- Duplicate values do not matter, we swap pointers not values

Pattern to remember

Many tree problems are just 'solve left, solve right, combine at the root'. Save one child in a temp variable before overwriting it.

3. Top K Frequent Elements

Medium | #347 | Array, Hash Table, Heap (Priority Queue), Bucket Sort, Counting
https://leetcode.com/problems/top-k-frequent-elements/

Problem

Given an integer array nums and an integer k, return the k values that appear most often. Any order is fine, and the answer is guaranteed to be unique.

Examples

Example 1	
Input	`nums = [1,1,1,2,2,3], k = 2`
Output	`[1,2]`
Why	1 appears 3 times and 2 appears 2 times, more than 3 which appears once.

Example 2	
Input	`nums = [1], k = 1`
Output	`[1]`
Why	Only one distinct value, so it is the answer.

Example 3	
Input	`nums = [1,2,1,2,1,2,3,1,3,2], k = 2`
Output	`[1,2]`
Why	1 and 2 appear 4 times each, 3 appears twice.

Intuition

A frequency can never be more than n. So instead of sorting values by frequency (O(n log n)), drop each value into bucket number f where f is its count. Then walk the buckets from the highest frequency down and pick values until we have k. This is bucket sort and runs in O(n).

Approach

1. Count every value in a HashMap cnt.
2. Make an array of lists b of size n + 1; b[f] holds values that appear exactly f times.
3. For each key x, add it to b[cnt.get(x)].
4. Walk f from n down to 1, copying values from b[f] into res until idx reaches k.
5. Return res.

Dry run on Example 1

- nums = [1,1,1,2,2,3], k = 2, n = 6
- cnt = {1:3, 2:2, 3:1}
- b[3] = [1], b[2] = [2], b[1] = [3], others null
- f = 6, 5, 4: b[f] is null, skip
- f = 3: res[0] = 1, idx = 1
- f = 2: res[1] = 2, idx = 2, loop condition idx < k fails
- Return [1,2]

Java solution

```
import java.util.*;

class Solution {
    public int[] topKFrequent(int[] nums, int k) {
```

```
        Map<Integer, Integer> cnt = new HashMap<>();
        for (int x : nums) cnt.put(x, cnt.getOrDefault(x, 0) + 1);
        List<Integer>[] b = new List[nums.length + 1];
        for (int x : cnt.keySet()) {
            int f = cnt.get(x);
            if (b[f] == null) b[f] = new ArrayList<>();
            b[f].add(x);
        }
        int[] res = new int[k];
        int idx = 0;
        for (int f = nums.length; f > 0 && idx < k; f--) {
            if (b[f] == null) continue;
            for (int x : b[f]) {
                if (idx == k) break;
                res[idx++] = x;
            }
        }
        return res;
    }
}
```

Complexity

Time: O(n) for counting and walking the buckets Space: O(n) for the map and buckets

Edge cases handled

- k equals the number of distinct values: return all of them
- All elements equal: one bucket at f = n
- Negative numbers work since the map keys are values, not indexes
- Several values in one bucket: the inner break stops exactly at k

Pattern to remember

When the key you sort by is bounded by n (like a count), bucket it by that key to beat O(n log n). A min-heap of size k is the other common answer.

4. Number of Islands

Medium | #200 | Array, Depth-First Search, Breadth-First Search, Union-Find, Matrix
https://leetcode.com/problems/number-of-islands/

Problem

You get an m x n grid of '1' (land) and '0' (water). Land cells that touch up, down, left or right form one island. Count the islands.

Examples

Example 1	
Input	`grid = [["1","1","1","1","0"],["1","1","0","1","0"],["1","1","0","0","0"],["0","0","0","0","0"]]`
Output	`1`
Why	All the land cells are connected, so there is one island.

Example 2	
Input	`grid = [["1","1","0","0","0"],["1","1","0","0","0"],["0","0","1","0","0"],["0","0","0","1","1"]]`
Output	`3`
Why	A 2x2 block at the top left, a single cell in the middle and a pair at the bottom right.

Intuition

Each time we find a land cell that has not been visited, we have found a new island. Flood fill from it (BFS) to mark every connected land cell as water, so the same island is never counted again. BFS with a queue is used instead of recursion because a 300 x 300 grid of all land would make recursive DFS go about 90000 calls deep and risk a stack overflow.

Approach

1. Set cnt = 0 and dirs = the four moves.
2. Scan every cell (i, j). Skip it if it is not '1'.
3. Found new land: cnt++, set grid[i][j] = '0' and push it into q.
4. While q is not empty: poll cur, try all 4 neighbours (x, y); if inside the grid and '1', set it to '0' and push it.
5. When the scan ends, return cnt.

Dry run on Example 1

- Example 1, m = 4, n = 5, cnt = 0
- (0,0) is '1': cnt = 1, mark it '0', q = [(0,0)]
- poll (0,0): push (1,0) and (0,1), both marked '0'
- poll (1,0): push (2,0) and (1,1); poll (0,1): push (0,2)
- poll (2,0): push (2,1); poll (1,1), (0,2): push (0,3); poll (2,1); poll (0,3): push (1,3); poll (1,3)
- q is empty, all 9 land cells are now '0'
- the rest of the scan only sees '0', so return cnt = 1

Java solution

```
import java.util.*;

class Solution {
    public int numIslands(char[][] grid) {
        int m = grid.length, n = grid[0].length, cnt = 0;
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        Queue<int[]> q = new ArrayDeque<>();
        for (int i = 0; i < m; i++) {
```

```
            for (int j = 0; j < n; j++) {
                if (grid[i][j] != '1') continue;
                cnt++;
                grid[i][j] = '0';
                q.add(new int[]{i, j});
                while (!q.isEmpty()) {
                    int[] cur = q.poll();
                    for (int[] d : dirs) {
                        int x = cur[0] + d[0], y = cur[1] + d[1];
                        if (x < 0 || y < 0 || x >= m || y >= n || grid[x][y] != '1') continue;
                        grid[x][y] = '0';
                        q.add(new int[]{x, y});
                    }
                }
            }
        }
        return cnt;
    }
}
```

Complexity

Time: O(m * n), each cell is pushed at most once Space: O(min(m, n)) for the BFS queue in practice, O(m * n) worst case

Edge cases handled

- All water: return 0
- All land: one island
- Checkerboard: every land cell is its own island (diagonals do not connect)
- 1 x 1 grid
- Note: the grid is modified in place; copy it first if the caller needs it

Pattern to remember

Counting connected components on a grid = scan + flood fill (BFS, DFS or Union-Find). Mark cells visited when you push them, not when you pop them, to avoid duplicates in the queue.

5. Minimum Window Substring

Hard | #76 | Hash Table, String, Sliding Window
https://leetcode.com/problems/minimum-window-substring/

Problem

Given strings s and t, find the shortest substring of s that contains every character of t, counting duplicates. Return "" if there is no such substring.

Examples

Example 1	
Input	`s = "ADOBECODEBANC", t = "ABC"`
Output	`"BANC"`
Why	BANC contains A, B and C and no shorter window does.

Example 2	
Input	`s = "a", t = "a"`
Output	`"a"`
Why	The whole string is the minimum window.

Example 3	
Input	`s = "a", t = "aa"`
Output	`""`
Why	t needs two 'a's but s has only one, so no window works.

Intuition

Use a sliding window [l, r]. cnt[c] stores how many more copies of c the window still needs (it goes negative for extra copies). need counts the total characters of t still missing. Grow r until need hits 0, meaning the window is valid; then shrink l as far as possible while it stays valid, recording the best length. Each index enters and leaves the window once, so it is O(m + n).

Approach

1. cnt[c]++ for every c in t; need = t.length(), l = 0, start = 0, min = MAX.
2. For each r: if cnt[s[r]] > 0 before decrementing, this char was needed, so need--. Then cnt[s[r]]--.
3. While need == 0: if r - l + 1 < min, save min and start = l.
4. Then remove s[l]: cnt[s[l]]++; if it becomes > 0, the window is now missing it, so need++. Move l forward.
5. Return "" if min is still MAX, else s.substring(start, start + min).

Dry run on Example 1

- s = ADOBECODEBANC, t = ABC: cnt[A]=cnt[B]=cnt[C]=1, need = 3
- r=0 A: needed, need = 2; r=1 D, r=2 O: extras, cnt goes to -1; r=3 B: need = 1; r=4 E: extra
- r=5 C: need = 0. Window ADOBEC (len 6): min = 6, start = 0. Remove A: cnt[A] = 1 > 0, need = 1, l = 1
- r=6 O, r=7 D, r=8 E: extras; r=9 B: cnt[B] was 0, extra, becomes -1
- r=10 A: need = 0. Window l=1..10 len 10, not better. Shrink: remove D, O, B (cnt[B] back to 0), E, l = 5; len 6 not better; remove C: cnt[C] = 1, need = 1, l = 6
- r=11 N: extra; r=12 C: need = 0. Window l=6..12 len 7, not better. Remove O, l = 7; len 6 not better; remove D, l = 8
- len 5 < 6: min = 5, start = 8. Remove E, l = 9; len 4 < 5: min = 4, start = 9. Remove B: cnt[B] = 1, need = 1, l = 10, stop

- Loop ends. Return s.substring(9, 13) = BANC

Java solution

```
class Solution {
    public String minWindow(String s, String t) {
        int[] cnt = new int[128];
        for (char c : t.toCharArray()) cnt[c]++;
        int need = t.length(), l = 0, start = 0, min = Integer.MAX_VALUE;
        for (int r = 0; r < s.length(); r++) {
            if (cnt[s.charAt(r)]-- > 0) need--;
            while (need == 0) {
                if (r - l + 1 < min) {
                    min = r - l + 1;
                    start = l;
                }
                if (++cnt[s.charAt(l++)] > 0) need++;
            }
        }
        return min == Integer.MAX_VALUE ? "" : s.substring(start, start + min);
    }
}
```

Complexity

Time: O(m + n), l and r each move at most m times Space: O(1), a fixed 128-size array

Edge cases handled

- t longer than s: need never reaches 0, return ""
- t has duplicates like "aa": cnt handles counts, not just presence
- Upper and lower case are different characters
- Answer at the very end of a long string, e.g. 100000 x's followed by "yz"

Pattern to remember

Variable sliding window: expand right until valid, then shrink left while still valid and record the answer. A single 'need' counter avoids comparing two whole frequency maps each step.